

TALLER DE LABORATORIO N.º 2

Mensajería con RabbitMQ

en contenedores Docker

Informe de evidencias

Integrantes	Derly Dayana Garcia Carrillo Orlando Rojas Narvaez
Docente	Juan Carlos Polania Cortes
Semestre / Periodo	IX · 2026-2
Repositorio	github.com/OrlandoRojasNa/taller02-rabbitmq-docker
Fecha	22 de septiembre de 2026

1. Descripción de la solución

Sistema de mensajería asíncrona completamente dockerizado. El equipo solo necesita **Docker**: el lenguaje, las librerías y el broker viven dentro de las imágenes.

```
Postman —POST /api/mensajes→ productor —AMQP→ [ rabbitmq : cola.sensores ] —AMQP→ consumidor → logs
```

Servicio	Tecnología	Función	Puerto publicado
rabbitmq	RabbitMQ 4.1 + management	Intermediario. La cola durable <code>cola.sensores</code> se crea al arrancar desde <code>definitions.json</code> .	15672 (consola)
productor	Java 21 · Spring Boot 3.5	API REST que valida el JSON y lo publica como mensaje persistente con confirmación del broker.	8080 (API)
consumidor	Java 21 · Spring Boot 3.5	Lee la cola con ACK manual y registra el contenido y la hora de recepción en sus logs.	ninguno

Cumplimiento de requisitos

Requisito	Implementación
Todo en contenedores	Dockerfiles multi-etapa: <code>maven:3.9-eclipse-temurin-21</code> compila y <code>eclipse-temurin:21-jre</code> ejecuta.
Un único comando	<code>docker compose up -d --build</code> levanta los tres servicios.
Cola durable existente al levantar	<code>load_definitions</code> de RabbitMQ con <code>durable: true</code> . Las aplicaciones también la declaran con los mismos atributos (idempotente).
POST + JSON, <code>status</code> booleano	<code>POST /api/mensajes</code> . La respuesta es <code>record Respuesta(boolean status, String message)</code> con <code>Content-Type: application/json</code> .
Retirar el mensaje solo después de procesarlo	<code>acknowledge-mode: manual: basicAck</code> después de registrar el mensaje, <code>prefetch: 1</code> .
Persistencia ante reinicios	Cola durable + mensajes <code>PERSISTENT</code> + volumen <code>rabbitmq_data</code> + <code>hostname</code> fijo.
Servicios por nombre, reintento de conexión	Host <code>rabbitmq</code> en la red de Docker. <code>depends_on: service_healthy</code> + reintentos automáticos de Spring AMQP.

Respuestas de la API

Caso	HTTP	Cuerpo
Mensaje publicado	200	<code>{"status": true, "message": "Mensaje encolado"}</code>
Cuerpo vacío, no JSON o mal formado	400	<code>{"status": false, "message": "Formato de mensaje inválido"}</code>
RabbitMQ no disponible	503	<code>{"status": false, "message": "No se pudo publicar el mensaje en la cola"}</code>

2. Instructivo para levantar el entorno

En un equipo que solo tenga Docker instalado:

```
git clone https://github.com/OrlandoRojasNa/taller02-rabbitmq-docker.git
cd taller02-rabbitmq-docker
docker compose up -d --build
```

Recurso	Acceso
Consola de RabbitMQ	http://localhost:15672 · usuario <code>lab</code> · contraseña <code>lab123</code>
Endpoint del productor	POST <code>http://localhost:8080/api/mensajes</code>
Logs del consumidor	<code>docker compose logs -f consumidor</code>
Colección de Postman	<code>postman/Taller02-RabbitMQ.postman_collection.json</code>

El instructivo completo, con los comandos de cada caso de prueba, está en el `README.md` del repositorio.

3. Evidencias del entorno

```
PS C:\cc-repos\taller02-rabbitmq-docker> docker compose ps
NAME                                COMMAND                                SERVICE    CREATED        STATUS        PORT
consumidor  sha256:9dc06ba60a8524b3e93d693a6b53b9e6b70cbb26847947f7b418f7a06d172ea "java -jar app.jar"  consumidor About a minute ago Up About a minute
producer    sha256:2af8b177bf4006cbf16cc28460f7d13a173a8c12cd78de0a1f6c080dc9e18868 "java -jar app.jar"  producer   About a minute ago Up About a minute    0.0.
0.0:8080->8080/tcp, [::]:8080->8080/tcp
rabbitmq    sha256:15b42c59481d4ac61a163b7417d749af0c0bc2b3449638ad318f8e9bb304dc42 "docker-entrypoint.s..." rabbitmq    About a minute ago Up About a minute (healthy) 0.0.
0.0:15672->15672/tcp, [::]:15672->15672/tcp

PS C:\cc-repos\taller02-rabbitmq-docker>
```

Figura 1. `docker compose ps` : los tres contenedores en ejecución, `rabbitmq` en estado *healthy*. Solo se publican los puertos 15672 (consola) y 8080 (API).

RabbitMQ

RabbitMQ 4.1.8

Erlang 27.3.4.18

Refreshed 2026-09-22 16:44:58

Refresh every 5 seconds

Virtual host All

Cluster rabbit@rabbitmq

User lab

Log out

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Queues

All queues (1)

Page 1 of 1 - Filter:

Regex ?

Displaying 1 item , page size up to: 100

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	cola.sensores	classic	D Args	running	0	0	0			

Add a new queue

HTTP API

Documentation

Tutorials

New releases

Commercial edition

Commercial support

Discussions

Discord

Plugins

GitHub

Figura 2. Consola de administración recién levantado el entorno: la cola `cola.sensores` ya existe, marcada con **D** (durable), sin haberla creado a mano.

4. Verificación con Postman

The screenshot shows the Postman interface for a POST request named "Enviar mensaje valido" to the endpoint `{{baseUrl}}/api/mensajes`. The request is successful, returning a **200 OK** status with a response time of 347 ms and a body size of 208 B. The response body is a JSON object: `{"status": true, "message": "Mensaje encolado"}`. The "Test Results" tab shows 4/4 tests passed, indicating that the `status` field is correctly a boolean.

Overview **POST Enviar mensaje valido** + No Environment Save Share

HTTP Taller 02 - RabbitMQ en Docker > **Enviar mensaje valido**

POST `{{baseUrl}}/api/mensajes` Send

Docs Params Authorization Headers 10 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 5 Test Results 4/4 200 OK • 347 ms • 208 B • Save Response

{ } JSON Preview Visualize

```
1 {
2   "status": true,
3   "message": "Mensaje encolado"
4 }
```

Figura 3. Mensaje válido: `200 OK`, `{"status": true, "message": "Mensaje encolado"}`. Tests 4/4 (incluye la verificación de que `status` es booleano).

Overview | POST Enviar mensaje valido | **POST Enviar mensaje invalido** | + | No Environment | Save | Share

Taller 02 - RabbitMQ en Docker > **Enviar mensaje invalido (JSON mal formado)**

POST | {{baseUrl}}/api/mensajes | Send

Docs | **Params** | Authorization | Headers 10 | Body | Scripts | Settings | Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body | Cookies | Headers 4 | Test Results 4/4 | 400 Bad Request • 17 ms • 192 B • Save Response

{ } JSON | Preview | Debug with AI

```
1 {
2   "status": false,
3   "message": "Formato de mensaje inválido"
4 }
```

Figura 4. Mensaje inválido (JSON mal formado): 400 Bad Request, {"status": false, "message": "Formato de mensaje inválido"}. Tests 4/4.

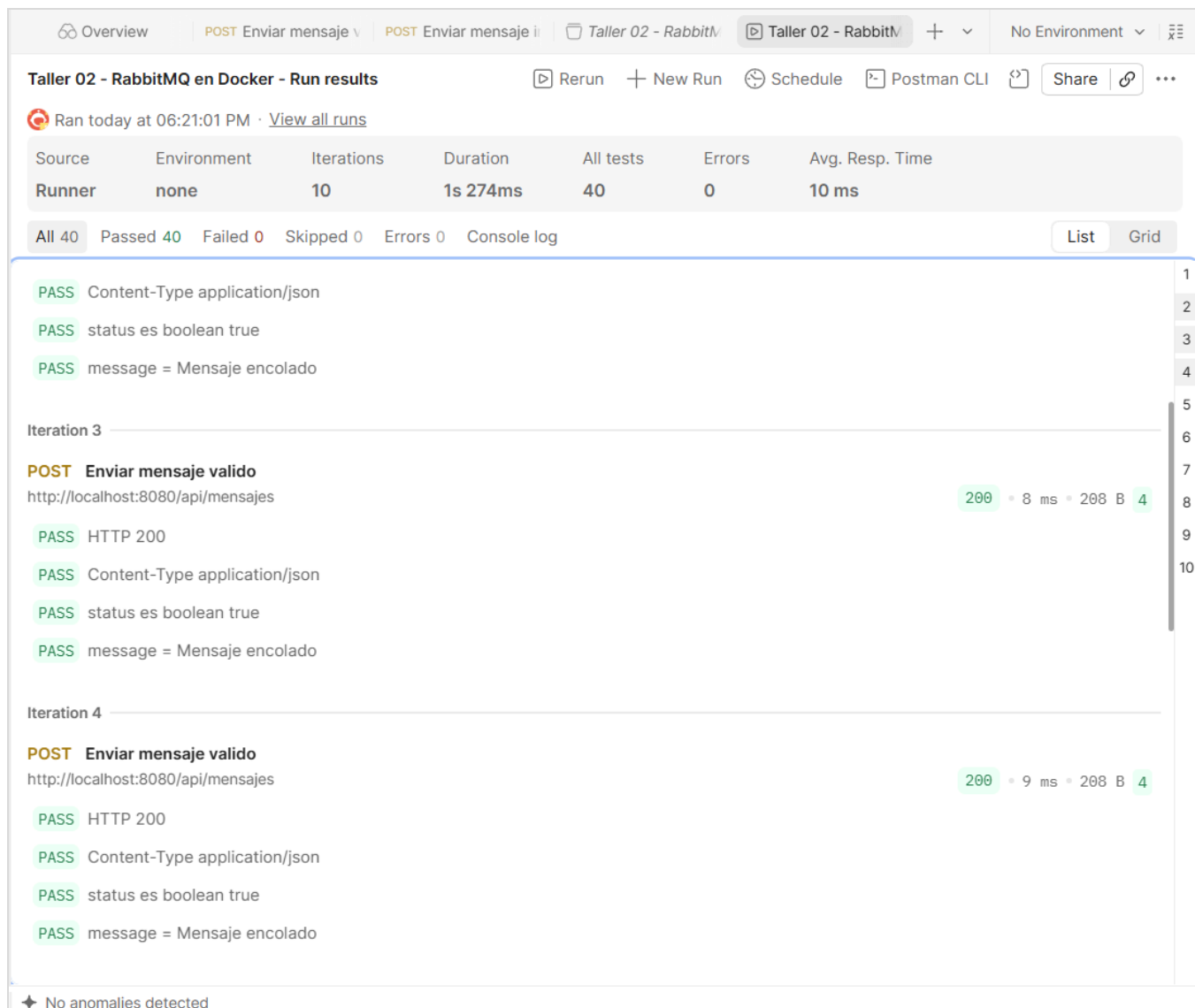


Figura 5. Collection Runner: 10 iteraciones de "Enviar mensaje valido". 40/40 tests aprobados, 0 errores.

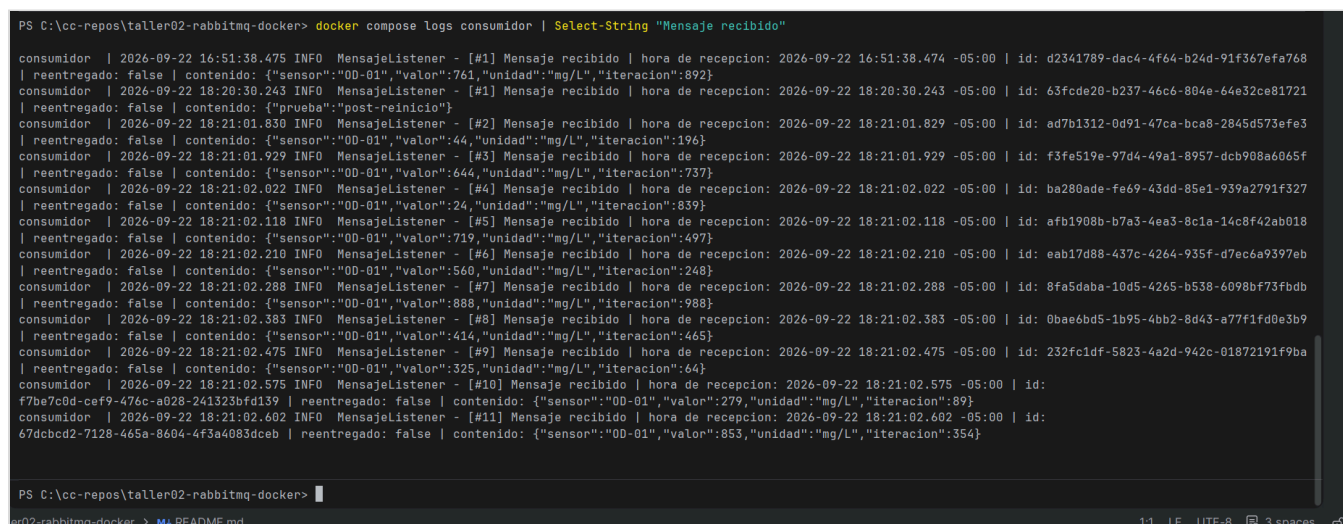


Figura 6. Log del consumidor: los 10 mensajes del Runner ([#2] a [#11] , 18:21:01–18:21:02) con su contenido y hora de recepción. [#1] corresponde a mensajes de prueba previos.

5. Casos del punto 7

Caso 1: consumidor detenido

Con `docker compose stop consumidor` se publicaron 5 mensajes. Se acumularon en la cola y, al ejecutar `docker compose start consumidor`, se consumieron todos.

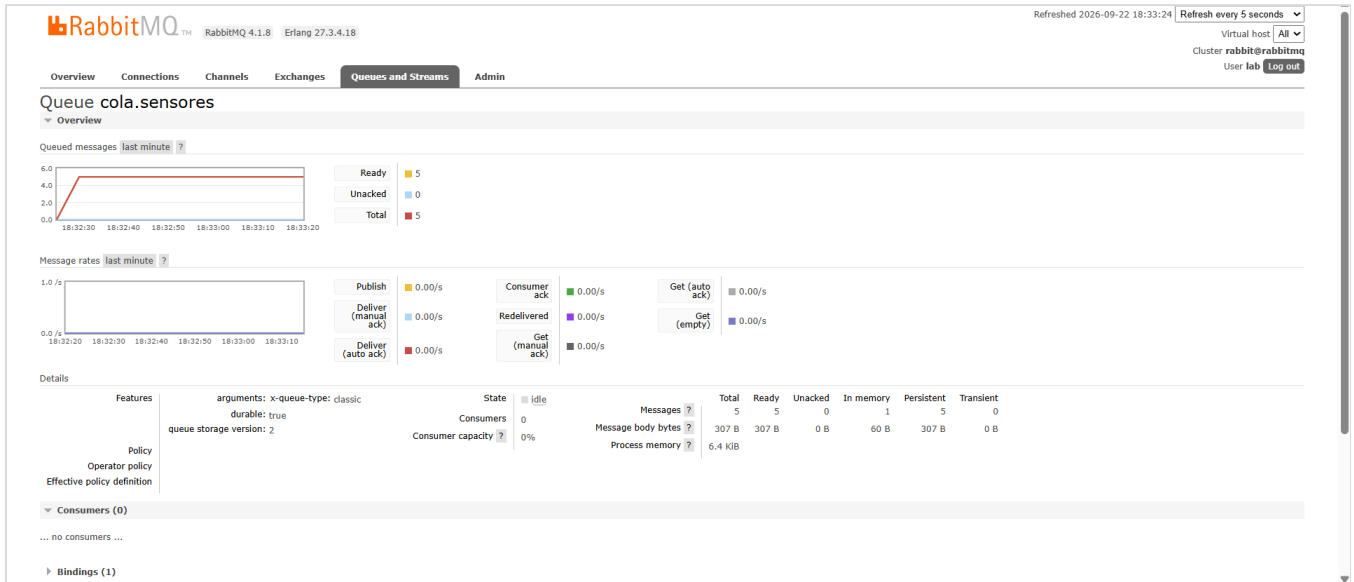


Figura 7. Antes. Consumidor detenido: **Ready = 5**, **Consumers (0)**. La cola es `durable: true` y los 5 mensajes son **Persistent**.

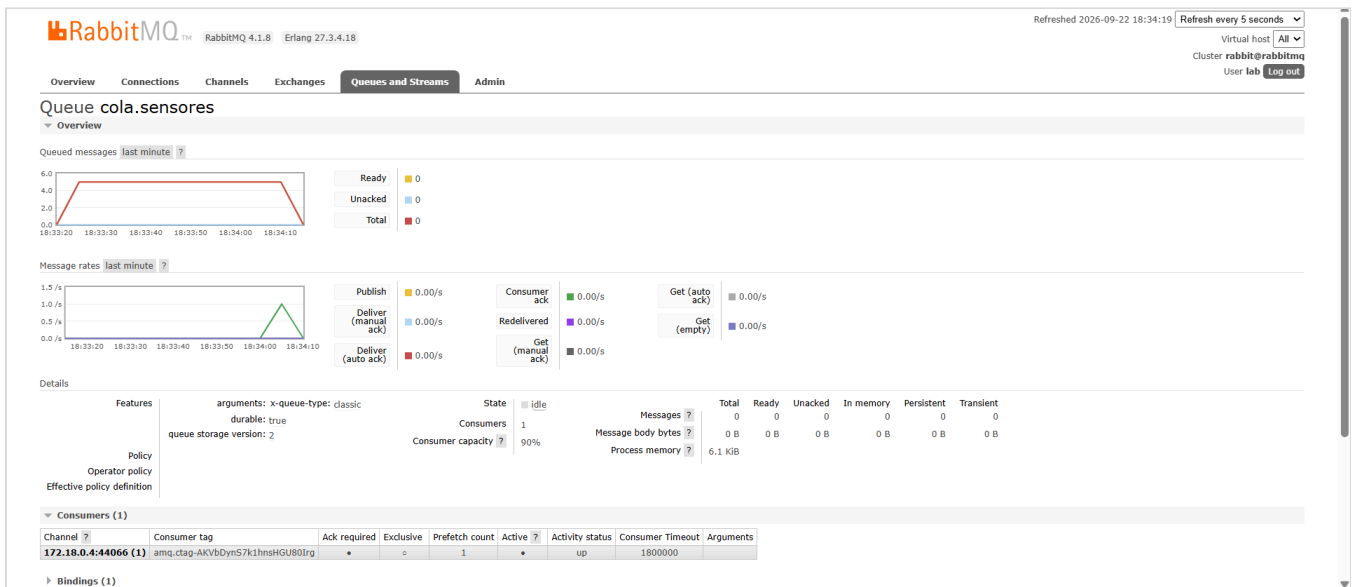


Figura 8. Después. Consumidor levantado: la gráfica baja de 5 a 0, pico de *Consumer ack*, **Consumers (1)** con **Ack required** (ACK manual) y **Prefetch count 1**.

```
PS C:\cc-repos\taller02-rabbitmq-docker> docker compose logs consumidor | Select-String "Mensaje recibido" | Select-Object -Last 5

consumidor | 2026-09-22 18:34:08.845 INFO Mensajelister - [#1] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.845 -05:00 | id: a7a9c8a3-93e4-471e-83c7-7acf109f69f5
| reentregado: false | contenido: {"sensor":"00-01","valor":1,"unidad":"mg/L","iteracion":232}
consumidor | 2026-09-22 18:34:08.852 INFO Mensajelister - [#2] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.852 -05:00 | id: b870f8a5-c217-4aa9-8726-ac09cb5ba783
| reentregado: false | contenido: {"sensor":"00-01","valor":874,"unidad":"mg/L","iteracion":268}
consumidor | 2026-09-22 18:34:08.854 INFO Mensajelister - [#3] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.854 -05:00 | id: fba42fd8-9952-4bd0-98c0-0cc00287bf31
| reentregado: false | contenido: {"sensor":"00-01","valor":246,"unidad":"mg/L","iteracion":683}
consumidor | 2026-09-22 18:34:08.856 INFO Mensajelister - [#4] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.856 -05:00 | id: cf7ba48f-8bce-4b75-90c2-293f5d456dc0
| reentregado: false | contenido: {"sensor":"00-01","valor":645,"unidad":"mg/L","iteracion":488}
consumidor | 2026-09-22 18:34:08.858 INFO Mensajelister - [#5] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.858 -05:00 | id: 02f33544-aa07-4c06-ae7d-70896f19a691
| reentregado: false | contenido: {"sensor":"00-01","valor":12,"unidad":"mg/L","iteracion":210}

PS C:\cc-repos\taller02-rabbitmq-docker>
```

Figura 9. Log del consumidor: los 5 mensajes acumulados, recibidos a las 18:34:08, apenas se levantó el consumidor.

Caso 2: reinicio de RabbitMQ

Con el consumidor detenido (`docker compose stop consumidor`) se publicaron varios mensajes desde Postman y se anotó el valor de *Ready*. Luego se reinició el broker con `docker compose restart rabbitmq`, se volvió a iniciar sesión en la consola y se comprobó que la cola tenía **la misma cantidad** de mensajes. Por último, `docker compose start consumidor` los procesó todos.

Los mensajes sobreviven porque se cumplen las cuatro condiciones a la vez:

Condición	Dónde se configura
La cola es durable (su definición se guarda en disco)	<code>rabbitmq/definitions.json</code> → <code>"durable": true</code>
Cada mensaje es persistente (se escribe en disco)	<code>PublicadorService</code> → <code>MessageDeliveryMode.PERSISTENT</code>
Los datos del nodo viven fuera del contenedor	<code>docker-compose.yml</code> → volumen <code>rabbitmq_data:/var/lib/rabbitmq</code>
El nombre del nodo no cambia entre reinicios	<code>docker-compose.yml</code> → <code>hostname: rabbitmq</code>

Además, el productor no necesita reiniciarse: Spring AMQP abre una conexión nueva en la siguiente publicación, y el consumidor reintenta conectarse cada 5 s.

Captura no incluida en esta entrega: 08-caso2-antes.png

Figura 10. Antes. Consumidor detenido, mensajes acumulados en `cola.sensores` (*Ready* > 0, **Consumers (0)**), justo antes de `docker compose restart rabbitmq`.

Captura no incluida en esta entrega: 08-caso2-despues.png

Figura 11. Después. Tras el reinicio del broker (la gráfica se reinicia porque las estadísticas no se conservan), la cola sigue existiendo y *Ready* muestra **la misma cantidad** de mensajes: no se perdió ninguno.

Caso 3: mensaje inválido

Si la API recibe un cuerpo que no es JSON válido, responde con `status` en `false` y no publica nada en la cola. Se anotó el total de `cola.sensores`, se envió *Enviar mensaje invalido (JSON mal formado)* desde Postman y se comprobó que el total no cambió.

La validación ocurre **antes** de tocar RabbitMQ: `PublicadorService.validar()` exige un objeto JSON con al menos un campo y sin contenido extra al final (`FAIL_ON_TRAILING_TOKENS`). Si falla, lanza `MensajeInvalidoException`, `ManejadorErrores` la convierte en `400` con `{"status": false,`

"message": "Formato de mensaje inválido"} y en el log del productor queda Mensaje rechazado, no se publica: ...

Cuerpo enviado	HTTP	¿Se publica?
{"sensor": "OD-01", "valor": } (mal formado)	400	No
esto no es json (texto plano)	400	No
(cuerpo vacío)	400	No
{"sensor": "OD-01"} basura (contenido extra)	400	No
[1, 2, 3] o {} (no es un objeto con campos)	400	No

Captura no incluida en esta entrega: 09-caso3-antes.png

Figura 12. Antes. Total de mensajes de cola.sensores antes de enviar el mensaje inválido.

Captura no incluida en esta entrega: 09-caso3-despues.png

Figura 13. Después. Postman recibe 400 Bad Request con "status": false y el total de la cola no cambia.

6. Entregables

Entregable	Ubicación en el repositorio
Código fuente del productor y del consumidor	<code>productor/src</code> , <code>consumidor/src</code>
Dockerfiles y Docker Compose	<code>productor/Dockerfile</code> , <code>consumidor/Dockerfile</code> , <code>rabbitmq/Dockerfile</code> , <code>docker-compose.yml</code>
Instructivo	<code>README.md</code>
Colección de Postman exportada	<code>postman/Taller02-RabbitMQ.postman_collection.json</code>
Capturas	<code>docs/evidencias/</code>
Declaración de uso de IA	<code>docs/DECLARACION_IA.md</code> y sección 7 de este informe

7. Declaración de uso de herramientas de inteligencia artificial

Herramienta: Claude Code (Anthropic), modelo Claude Opus 5.5, desde la aplicación de escritorio de Claude.

Finalidad:

- Leer el enunciado del taller, extraer los requisitos y revisar que el documento no tuviera texto oculto.
- Generar el proyecto completo: `docker-compose.yml` , Dockerfiles, configuración de RabbitMQ, el código del productor y del consumidor (Java 21 + Spring Boot) y las pruebas unitarias del productor.
- Generar la colección de Postman y el instructivo (`README.md`).
- Crear el repositorio en GitHub, hacer los commits y agregar a la integrante como colaboradora.
- Guiar paso a paso la ejecución de las pruebas y la toma de capturas, y armar este informe.
- Diagnosticar dos problemas durante la ejecución: una caída momentánea de red al descargar imágenes de Docker Hub y Docker Desktop, que no se inició después de reiniciar el equipo.
- Redactar la documentación de los casos 2 y 3 del punto 7 (procedimiento, explicación y pies de figura). Antes de redactarla, la herramienta ejecutó ambos casos en un entorno de pruebas propio para comprobar el comportamiento. Las capturas de este informe las tomaron los integrantes en su equipo.

Qué se modificó del resultado: no se modificó el código generado por la herramienta. El trabajo de los integrantes sobre el resultado fue:

- Instalar Docker Desktop y levantar el entorno completo con `docker compose up -d --build` .
- Importar la colección en Postman y ejecutar las peticiones (mensaje válido, inválido y el Runner con 10 mensajes).
- Ejecutar los casos del punto 7 y verificar los resultados en la consola de administración y en los logs del consumidor.
- Tomar las capturas de evidencia.

Los integrantes respondemos por el funcionamiento del proyecto entregado.